NETPark Smart Parking Management System

Assigned By: Nikita mam

---

1. Object-Oriented Analysis

Possible Classes and Objects

- **User**: Represents a general user or driver using the system.
- **Admin**: Represents the system administrator managing the platform.
- **Slot**: Represents a physical parking slot available for booking.
- **Booking**: Represents a reservation made by a user for a slot.
- **Transaction**: Represents a financial record (wallet recharge or booking payment).

Attributes and Methods

- **User**
  - *Attributes*: id, name, email, phone, password, walletBalance, role, isVerified
  - *Methods*: register(), login(), updateProfile(), addWalletBalance()
- **Admin**
  - *Methods*: manageSlots(), manageUsers(), viewReports()
- **Slot**
  - *Attributes*: id, name, location, basePrice, status
  - *Methods*: addSlot(), updateStatus(), calculateDynamicPrice()
- **Booking**
  - *Attributes*: bookingId, userId, slotId, startTime, endTime, totalAmount, status
  - *Methods*: createBooking(), cancelBooking(), completeBooking()
- **Transaction**
  - *Attributes*: transactionId, userId, type, amount, status, timestamp
  - *Methods*: initiateTransaction(), verifyTransaction()

Associations & Cardinality

- **User to Booking**: One-to-Many (One user can make multiple bookings)
- **User to Transaction**: One-to-Many (One user can initiate multiple transactions)
- **Slot to Booking**: One-to-Many (One slot can have multiple sequential bookings)

UML Class Diagram

 UML Class Diagram

---

2. Usecase Analysis

Identify Usecases and Actors

Actors:

- **User**: Registers, logs in, adds money to wallet, books slots, and views history.
- **Admin**: Manages slots, views analytics, and manages users.
- **Payment System**: External entity handling wallet transactions.

Usecases:

- Register & Login
- Search & Book Slot
- Add Money to Wallet
- View Booking History
- Manage Parking Slots
- Monitor Analytics
- Manage Users

Usecase Diagram

 Usecase Diagram